

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	08 - 25 - 26	DATE SUBMITTED:	08 - 26 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 1	Familiarization for the new normal Lab
1. PROCEDURE OUTPUT	
Task 1: Run elevated ad hoc commands 1. Locally, we use the command <code>sudo apt update</code> when we want to download package information from all configured resources. The sources are often defined in <code>/etc/apt/sources.list</code> file and other files located in <code>/etc/apt/sources.list.d/</code> directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command: <code>ansible all -m apt -a update_cache=true</code> What is the result of the command? Is it successful? <i>The command failed because installing packages, especially on remote machines, requires admin privileges.</i> OUTPUT <pre>[ERROR]: Task failed: Module failed: Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied) Origin: <ad hoc 'apt' task> {'action': 'apt', 'args': {'update_cache': 'true'}, 'timeout': 0, 'async_val': 0, 'poll': 15} ubuntu2 FAILED! => { "ansible_facts": { "discovered_interpreter_python": "/usr/bin/python3.14" }, "changed": false, "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)" }</pre>	

```
ubuntu1 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
/var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
}
```

Try editing the command and add something that would elevate the privilege. Issue the command ***ansible all -m apt -a update_cache=true --become --ask-become-pass***. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The ***update_cache=true*** is the same thing as running `sudo apt update`. The ***--become*** command elevates the privileges and the ***--ask-become-pass*** asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is

CHANGED compared to the first command, which is **FAILED**.

```
ubuntu1 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
/var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
}

BECOME password:
ubuntu1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1786975175,
  "cache_updated": true,
  "changed": true
}
```

OUTPUT

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible ubuntu -i inventory.yaml
l -m apt -a update_cache=true --become --ask-become-pass
BECOME password:
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ubuntu2 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": true,
  "changed": true
}
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ubuntu1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": true,
  "changed": true
}
santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

- Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just change the module part in 1.1 instruction. Here is the command: ***ansible all -m apt -a name=vim-nox --become --ask-become-pass***. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

OUTPUT

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible ubuntu -i inventory
l -m apt -a name=vim-nox --become --ask-become-pass
BECOME password:
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/
python3.14', but future installation of another Python interpreter could cause
ferent interpreter to be discovered. See https://docs.ansible.com/ansible-core
/reference\_appendices/interpreter\_discovery.html for more information.
ubuntu1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading
information...\nSolving dependencies...\nThe following additional packages w
installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby
y3.3\n libsodium23 rake ruby ruby-csv ruby-did-you-mean ruby-net-telnet\n r
by2-keywords ruby-rubygems ruby-sdbm ruby-webrick ruby-xmlrpc ruby3.3\n ruby
ntegration vim-common vim-runtime vim-tiny xxd\nSuggested packages:\n apache
/reference_appendices/interpreter_discovery.html for more information.
ubuntu2 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading
information...\nSolving dependencies...\nThe following additional packages w
installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby
y3.3\n libsodium23 rake ruby ruby-csv ruby-did-you-mean ruby-net-telnet\n r
by2-keywords ruby-rubygems ruby-sdbm ruby-webrick ruby-xmlrpc ruby3.3\n ruby
ntegration vim-common vim-runtime vim-tiny xxd\nSuggested packages:\n apache
ghhttpd | httpd ri ruby-dev bundler cscope vim-doc\nThe following NEW packages
be installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 librub
```

- Verify that you have installed the package in the remote servers. Issue the command *which vim* and the command *apt search vim-nox* respectively. Was the command successful?

```
server1@server1:~$ which vim
/usr/bin/vim
server1@server1:~$ apt search vim-nox
vim-nox/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [installed]
Vi Improved - enhanced vi editor - with scripting languages support

vim-tiny/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [installed]
Vi Improved - enhanced vi editor - compact version
```

```
server2@server2:~$ which vim
/usr/bin/vim
server2@server2:~$ apt search vim-nox
vim-nox/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [installed]
Vi Improved - enhanced vi editor - with scripting languages support

vim-tiny/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [installed]
Vi Improved - enhanced vi editor - compact version
```

lines 1-6/6 (END)

Vim was successfully installed on both servers.

- Check the logs in the servers using the following commands: `cd /var/log`. After this, issue the command `ls`, go to the folder `apt` and open `history.log`. Describe what you see in the history.log.

OUTPUT

```
server1@server1:~$ cd /var/log
server1@server1:/var/log$ ls
README bootstrap.log dmesg.1.gz journal syslog.2.gz
alternatives.log btmp dmesg.2.gz kern.log sysstat
alternatives.log.1 btmp.1 dmesg.3.gz kern.log.1 ubuntu-advantage-apt-hook.log
apport.log chrony dmesg.4.gz kern.log.2.gz ufw.log
apt cloud-init-output.log dpkg.log lastlog ufw.log.1
auth.log cloud-init.log dpkg.log.1 openvpn ufw.log.2.gz
auth.log.1 cups fontconfig.log private unattended-upgrades
auth.log.2.gz cups-browsed gdm3 speech-dispatcher vboxpostinstall.log
boot.log dist-upgrade gpu-manager.log sssd wtmp
boot.log.1 dmesg hp syslog
boot.log.2 dmesg.0 installer syslog.1

server1@server1:/var/log$ cd apt
server1@server1:/var/log/apt$ cat history.log

Start-Date: 2026-08-24 10:19:02
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=-force-confdef -o Dpkg::Options::=-force-confold -o Dpkg::Lock::Timeout=60 install vim-nox=2:9.1.2141-1ubuntu4.8
Requested-By: server1 (1000)
Install: ruby-ruby2-keywords:amd64 (0.0.5-1build1, automatic), ruby-csv:amd64 (3.3.5-1, automatic), ruby-sdbm:amd64 (1.0.0-5build6, automatic), liblua5.1-0:amd64 (5.1.5-12, automatic), fonts-lato:amd64 (2.015-1build1, automatic), libruby:amd64 (1:3.3build1, automatic), ruby-net-telnet:amd64 (0.2.0-1build1, automatic), rubygems-integration:amd64 (1.19build1, automatic), libruby3.3:amd64 (3.3.8-2ubuntu3.1, automatic), rake:amd64 (13.3.1-1, automatic), libsodium23:amd64 (1.0.18-2, automatic), vim-nox:amd64 (2:9.1.2141-1ubuntu4.8), ruby:amd64 (1:3.3build1, automatic), vim-runtime:amd64 (2:9.1.2141-1ubuntu4.8, automatic), ruby3.3:amd64 (3.3.8-2ubuntu3.1, automatic), ruby-did-you-mean:amd64 (2.0.0-1, automatic), libjs-jquery:amd64 (3.7.1+dfsg+~3.5.33-1build1, automatic), ruby-rubygems:amd64 (3.6.7-2ubuntu2, automatic), javascript-common:amd64 (12+nmu1build1, automatic), ruby-xmlrpc:amd64 (0.3.3-2build1, automatic), ruby-webrick:amd64 (1.9.2-1, automatic)
Upgrade: xxd:amd64 (2:9.1.2141-1ubuntu4.6, 2:9.1.2141-1ubuntu4.8), vim-common:amd64 (2:9.1.2141-1ubuntu4.6, 2:9.1.2141-1ubuntu4.8), vim-tiny:amd64 (2:9.1.2141-1ubuntu4.6, 2:9.1.2141-1ubuntu4.8)
End-Date: 2026-08-24 10:19:19
server1@server1:/var/log/apt$
```

The history logs show that VIM was installed.

3. This time, we will install a package called snapd. Snap is pre-installed in the Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

- a. Issue the command:

ansible all -m apt -a name=snapd --become --ask-become-pass

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible ubuntu -i inventory.yaml -m
apt -a name=snapd --become --ask-become-pass
BECOME password:
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/pytho
n3.14', but future installation of another Python interpreter could cause a different i
nterpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_a
ppendices/interpreter_discovery.html for more information.
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": false,
  "changed": false
}
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/pytho
n3.14', but future installation of another Python interpreter could cause a different i
nterpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_a
ppendices/interpreter_discovery.html for more information.
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": false,
  "changed": false
}
santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

It was successful. Since the snapd package is pre-installed in an Ubuntu system, it simply checked and showed the update status.

- b. Now, try to issue this command:

ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass

Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

The output is the same as the previous command. It shows the latest state of the snapd package.

OUTPUT

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible ubuntu -i inventory
.yaml -m apt -a "name=snapd state=latest" --become --ask-become-pass
BECOME password:
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/b
in/python3.14', but future installation of another Python interpreter could cau
se a different interpreter to be discovered. See https://docs.ansible.com/ansib
le-core/2.20/reference_appendices/interpreter_discovery.html for more informati
on.
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": false,
  "changed": false
}
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/b
in/python3.14', but future installation of another Python interpreter could cau
se a different interpreter to be discovered. See https://docs.ansible.com/ansib
le-core/2.20/reference_appendices/interpreter_discovery.html for more informati
on.
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "cache_update_time": 1787566649,
  "cache_updated": false,
  "changed": false
}
```

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of Ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we used in the previous activities (**CPE232_yourname**). Issue the command **nano**

install_apache.yml. This will create a playbook file called *install_apache.yml*. The .yml is the basic standard extension for playbook files.

When the editor appears, type the following:

```
---
- hosts: all #change if needed
  become: true
  tasks:

  - name: install apache2 package
    apt:
      name: apache2
```

Make sure to save the file. Take note also of the alignments of the texts. Use TAB instead of SPACE BAR for spacing.

2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml*. Describe the result of this command.

```
TASK [Install Apache2 package] *****
*
changed: [ubuntu1]
changed: [ubuntu2]

PLAY RECAP *****
*
centos1      : ok=0    changed=0    unreachable=1    failed=0
skipped=0    rescued=0    ignored=0
ubuntu1     : ok=2    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
ubuntu2     : ok=2    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

It failed to install apache on the CentOS server because it uses yum/dnf instead of apt.

Adding these lines to my playbook allowed the installation of apache on centos as well.:


```
---
- hosts: linuxservers
  become: true
  tasks:
    - name: Install Apache2 package Ubuntu
      apt:
        name: apache2

      when: ansible_os_family == "Debian" # for Ubuntu

    - name: Install Apache2 package CentOS
      dnf:
        name: httpd
      when: ansible_os_family == "RedHat" # for CentOS
```

```
TASK [Install Apache2 package Ubuntu] *****
*
```

```
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
```

```
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to `True` is deprecated, to p-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
```

```
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:9:13
```

```
7         name: apache2
```

```
8
```

```
9         when: ansible_os_family == "Debian" # for Ubuntu
           ^ column 13
```

```
Use `ansible_facts["fact_name"]` (no `ansible_` prefix) instead.
```

```
skipping: [centos1]
```

```
ok: [ubuntu2]
```

```
ok: [ubuntu1]
```

```
TASK [Install Apache2 package CentOS] *****
*
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, to
p-level facts will not be auto injected after the change. This feature will be
removed from ansible-core version 2.24.
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:14:13

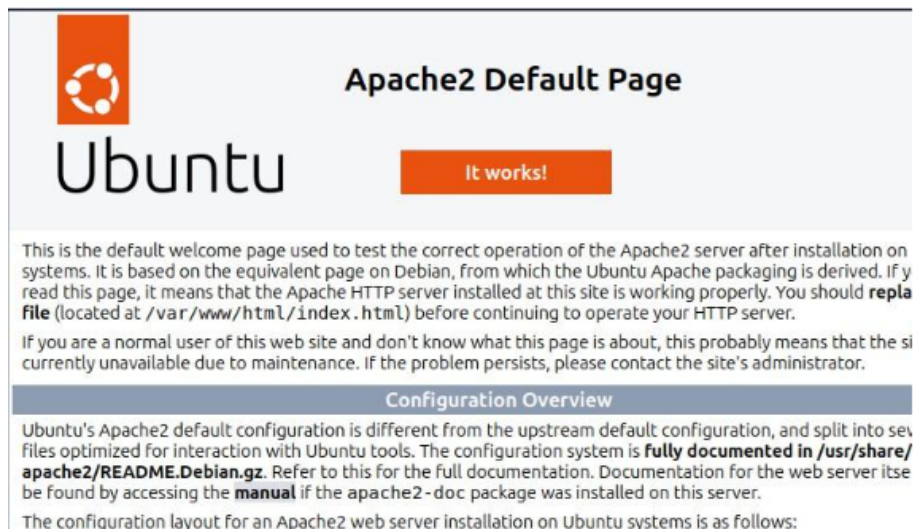
12     dnf:
13         name: httpd
14         when: ansible_os_family == "RedHat" # for CentOS
              ^ column 13

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.

skipping: [ubuntu1]
skipping: [ubuntu2]
```

```
PLAY RECAP *****
*
centos1      : ok=2    changed=1    unreachable=0    failed=0
skipped=1    rescued=0    ignored=0
ubuntu1     : ok=2    changed=0    unreachable=0    failed=0
skipped=1    rescued=0    ignored=0
ubuntu2     : ok=2    changed=0    unreachable=0    failed=0
skipped=1    rescued=0    ignored=0
```

- To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace** the file (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

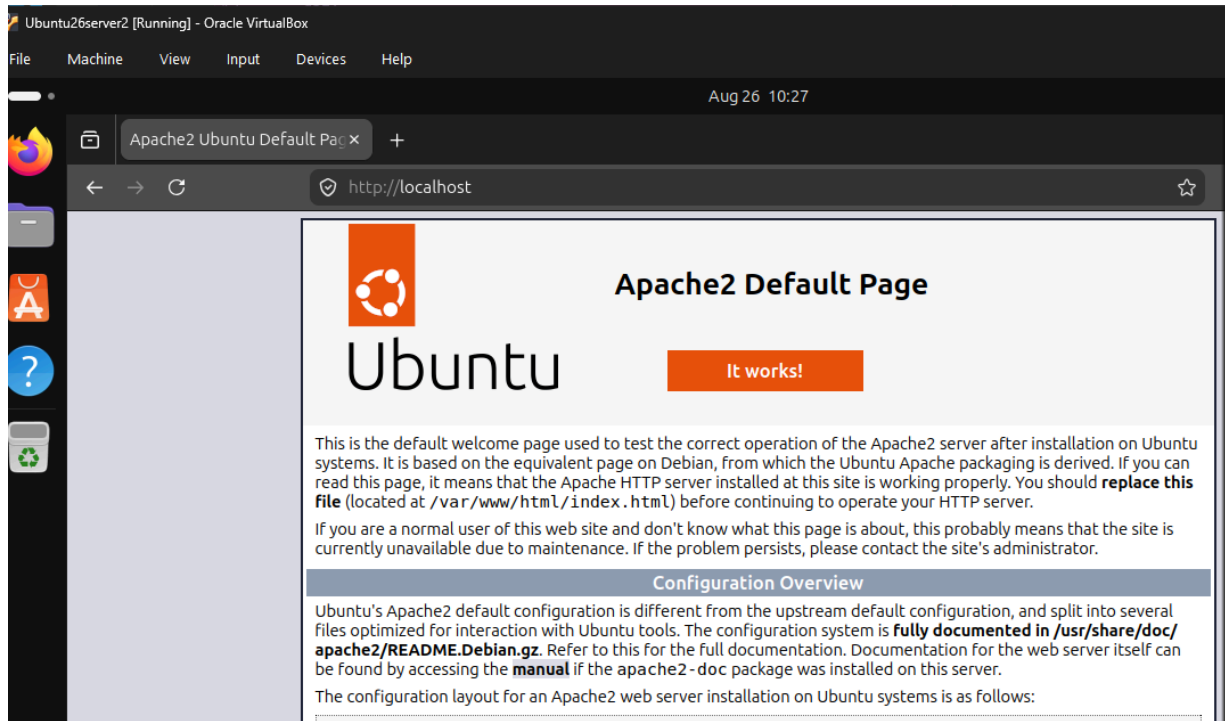
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

OUTPUT



4. Try to edit the `install_apache.yml` and change the name of the package to any name that will not be recognized. What is the output?

```
16
17     - name: Install unknown package
18       apt:
19         name: unkown_package
20         when: ansible_os_family == "Debian" # for Ubuntu
21
```

```
[ERROR]: Task failed: Module failed: No package matching 'unkown_package' is available
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:17:7

15     #when: ansible_os_family == "RedHat" # for CentOS
16
17     - name: Install unknown package
      ^ column 7

fatal: [ubuntu2]: FAILED! => {"changed": false, "msg": "No package matching 'unkown_package' is available"}
fatal: [ubuntu1]: FAILED! => {"changed": false, "msg": "No package matching 'unkown_package' is available"}

PLAY RECAP *****
*
centos1      : ok=1    changed=0    unreachable=0    failed=0
skipped=1    rescued=0    ignored=0
ubuntu1     : ok=1    changed=0    unreachable=0    failed=1
skipped=0    rescued=0    ignored=0
ubuntu2     : ok=1    changed=0    unreachable=0    failed=1
skipped=0    rescued=0    ignored=0
```

The output showed that it attempted to install an unknown package but failed since it doesn't exist.

5. This time, we are going to put additional tasks into our playbook. Edit the *install_apache.yml*. As you can see, we are now adding an additional command, which is the *update_cache*. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

```
---
- hosts: all #change if needed
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes

    - name: install apache2 package
      apt:
        name: apache2
```

Save the changes to this file and exit.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```

1  ---
2  - hosts: linuxservers
3    become: true
4    tasks:
5
6      - name: Update Repository Index Ubuntu
7        apt:
8          update_cache: yes
9          when: ansible_os_family == "Debian" # for Ubuntu
10
11     - name: Update Repository Index CentOS
12       dnf:
13         update_cache: yes
14         when: ansible_os_family == "RedHat" # for CentOS
15

```

```

TASK [Update Repository Index Ubuntu] *****
*
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to `True` is deprecated, to p-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:9:13
7      apt:
8        update_cache: yes
9        when: ansible_os_family == "Debian" # for Ubuntu
           ^ column 13

Use `ansible_facts["fact_name"]` (no `ansible_` prefix) instead.

```

```
TASK [Update Repository Index CentOS] *****
*
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to `True` is deprecated, to
p-level facts will not be auto injected after the change. This feature will be
removed from ansible-core version 2.24.
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:14:13

12     dnf:
13         update_cache: yes
14         when: ansible_os_family == "RedHat" # for CentOS
           ^ column 13

Use `ansible_facts["fact_name"]` (no `ansible_` prefix) instead.

skipping: [ubuntu1]
skipping: [ubuntu2]
ok: [centos1]
```

```
PLAY RECAP *****
*
centos1      : ok=3    changed=0    unreachable=0    failed=0
skipped=2   rescued=0    ignored=0
ubuntu1     : ok=3    changed=1    unreachable=0    failed=0
skipped=2   rescued=0    ignored=0
ubuntu2     : ok=3    changed=1    unreachable=0    failed=0
skipped=2   rescued=0    ignored=0
```

Yes, it changed one thing on all servers by issuing an update command without upgrading.

7. Edit again the `install_apache.yml`. This time, we are going to add PHP support for the apache package we installed earlier.

```

---
- hosts: all #change if needed
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes

    - name: install apache2 package
      apt:
        name: apache2

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php

```

Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```

26     - name: Add PHP support for Apache Ubuntu
27       apt:
28         name: libapache2-mod-php
29       when: ansible_os_family == "Debian" # for Ubuntu
30
31     - name: Add PHP support for Apache CentOS
32       dnf:
33         name: php
34       when: ansible_os_family == "RedHat" # for CentOS
35

```



```
TASK [Add PHP support for Apache Ubuntu] *****
*
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, to
p-level facts will not be auto injected after the change. This feature will be
removed from ansible-core version 2.24.
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:29:13

27     apt:
28         name: libapache2-mod-php
29         when: ansible_os_family == "Debian" # for Ubuntu
           ^ column 13

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.

skipping: [centos1]
changed: [ubuntu1]
changed: [ubuntu2]
```

```
TASK [Add PHP support for Apache CentOS] *****
*
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, to
p-level facts will not be auto injected after the change. This feature will be
removed from ansible-core version 2.24.
Origin: /home/santiago/CPE212-Santiago/ansible-lab/install_apache.yml:34:13

32     dnf:
33         name: php
34         when: ansible_os_family == "RedHat" # for CentOS
           ^ column 13

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.

skipping: [ubuntu1]
skipping: [ubuntu2]
changed: [centos1]
```

```
PLAY RECAP *****
*
centos1      : ok=4    changed=1    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0
ubuntu1     : ok=4    changed=2    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0
ubuntu2     : ok=4    changed=2    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0
```

The command in the playbook successfully downloaded the corresponding PHP packages on both the Ubuntu and CentOS systems.

9. Finally, make sure that we are in sync with GitHub. Provide the link to your GitHub repository.

<https://github.com/owen-s-dotdev/CPE212-Santiago.git>

The main changes are found in the inventory.yaml and install_apache.yaml file. The machines used are my home machines.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?

A playbook simplifies ansible tasks by compiling all of them into one single executable file. Instead of relying on ad-hoc commands for individual instructions, we can define multiple in a single playbook. This simplifies server management and allows us to reuse them whenever needed.

2. Summarize what we have done on this activity.

In this activity, we explored ansible capabilities using ad-hoc commands and playbooks. In the ad-hoc section, I learned how ansible can be used to execute privileged commands by using the become keyword to elevate privileges. With that, I was able to install, update, and check the status of various packages on remote servers. After exploring privilege escalation, I moved on to practicing how to use a simple playbook for installing Apache. Here, I learned how to structure my playbooks to install various packages on both Ubuntu and CentOS systems.

4. CONCLUSION/ANALYSIS/LEARNINGS

In conclusion, this activity explores the basic functionalities and capabilities of Ansible for server management. I learned how to use ad-hoc commands for privilege escalation, allowing me to install packages on remote servers. Then, I created a simple playbook that automates remote installation of Apache and PHP support on both Ubuntu and CentOS devices. Most importantly, I learned how to structure my playbooks to install packages on different distros, allowing me to install Apache on both Ubuntu and CentOS using the same playbook. This further helped in improving my skills and capabilities in server management since it covers the basics of automation and privileged remote access.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____



COMPUTER
ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.